

LAB PROGRAM 1

Implementing basic array operations: insertion, deletion, searching, and traversal.

#include <stdio.h>

```
int main() {
    int arr[100], n, i, pos, value, choice, found;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    printf("Enter elements:\n");
    for(i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }
    do {
        printf("\n--- MENU ---\n");
        printf("1. Traversal\n2. Insertion\n3. Deletion\n4. Searching\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);
        switch(choice) {
            case 1: // Traversal
                printf("Array elements are: ");
                for(i = 0; i < n; i++) {
                    printf("%d ", arr[i]);
                }
                break;

            case 2: // Insertion
                printf("Enter position and value: ");
                scanf("%d %d", &pos, &value);

                for(i = n; i >= pos; i--) {
                    arr[i] = arr[i - 1];
                }
                arr[pos - 1] = value;
                n++;
                break;

            case 3: // Deletion
                printf("Enter position to delete: ");
                scanf("%d", &pos);
                for(i = pos - 1; i < n - 1; i++) {
                    arr[i] = arr[i + 1];
                }
                n--;
                break;

            case 4: // Searching
```

```

        printf("Enter element to search: ");
        scanf("%d", &value);
        found = 0;
        for(i = 0; i < n; i++) {
            if(arr[i] == value) {
                printf("Element found at position %d\n", i + 1);
                found = 1;
                break;
            }
        }

        if(found == 0)
            printf("Element not found\n");
        break;
    case 5:
        printf("Exiting...\n");
        break;
    default:
        printf("Invalid choice!\n");
    }
} while(choice != 5);
return 0;
}

```

Output-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p1.c -o p1
PS D:\DS PROGRAMS\LABPROGRAMS> ./p1
Enter number of elements: 5
Enter elements:
4 9 8 6 2

--MENU--
1.Traversal
2.Insertion
3.Deletion
4.Searching
5.Exit
Enter your choice: 2
Enter position and value: 2 10
Updated array: 4 10 9 8 6 2

--MENU--
1.Traversal
2.Insertion
3.Deletion
4.Searching
5.Exit
Enter your choice: 5
Exiting...
PS D:\DS PROGRAMS\LABPROGRAMS> 

```

LAB PROGRAM 2

#Representing a sparse matrix using triplet form and perform transpose operation.
#include <stdio.h>

```

int main() {

```

```

int matrix[10][10], triplet[50][3], transpose[50][3];
int i, j, m, n, k = 1;

printf("Enter rows and columns: ");
scanf("%d %d", &m, &n);

printf("Enter matrix elements:\n");
for(i = 0; i < m; i++) {
    for(j = 0; j < n; j++) {
        scanf("%d", &matrix[i][j]);
    }
}

// Convert to triplet form
triplet[0][0] = m;
triplet[0][1] = n;
triplet[0][2] = 0;

for(i = 0; i < m; i++) {
    for(j = 0; j < n; j++) {
        if(matrix[i][j] != 0) {
            triplet[k][0] = i;
            triplet[k][1] = j;
            triplet[k][2] = matrix[i][j];
            k++;
            triplet[0][2]++;
        }
    }
}

printf("\nTriplet Representation:\n");
for(i = 0; i < k; i++) {
    printf("%d %d %d\n", triplet[i][0], triplet[i][1], triplet[i][2]);
}

// Transpose
transpose[0][0] = triplet[0][1];
transpose[0][1] = triplet[0][0];
transpose[0][2] = triplet[0][2];

k = 1;

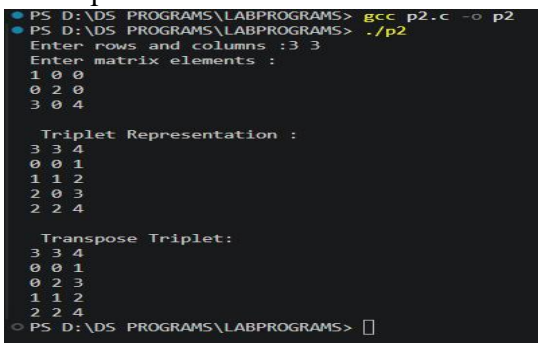
for(i = 0; i < n; i++) {
    for(j = 1; j <= triplet[0][2]; j++) {
        if(triplet[j][1] == i) {
            transpose[k][0] = triplet[j][1];

```

```

        transpose[k][1] = triplet[j][0];
        transpose[k][2] = triplet[j][2];
        k++;
    }
}
}
printf("\nTranspose Triplet:\n");
for(i = 0; i < k; i++) {
    printf("%d %d %d\n", transpose[i][0], transpose[i][1], transpose[i][2]);
}
return 0;
}
Output-

```



```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p2.c -o p2
PS D:\DS PROGRAMS\LABPROGRAMS> ./p2
Enter rows and columns :3 3
Enter matrix elements :
1 0 0
0 2 0
3 0 4

Triplet Representation :
3 3 4
0 0 1
1 1 2
2 0 3
2 2 4

Transpose Triplet:
3 3 4
0 0 1
0 2 3
1 1 2
2 2 4
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 3A

#Implementing stack operations using arrays and linked lists.

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// Push
```

```
void push(int value) {
    if (top == MAX - 1)
        printf("Stack Overflow\n");
    else {
        top++;
        stack[top] = value;
        printf("Pushed %d\n", value);
    }
}
```

```
// Pop
```

```
void pop() {
    if (top == -1)
        printf("Stack Underflow\n");
}
```

```

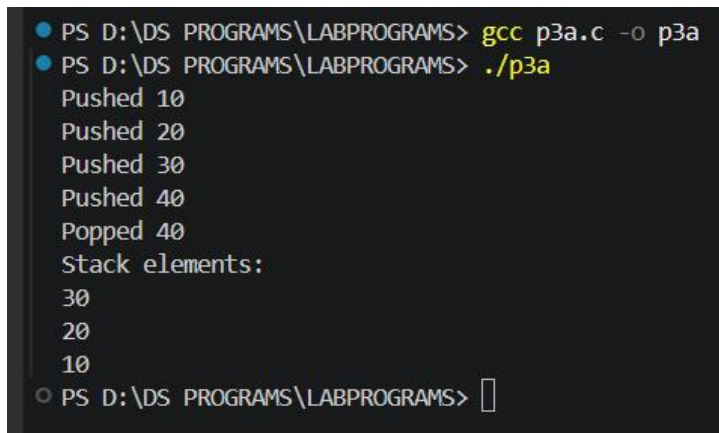
    else {
        printf("Popped %d\n", stack[top]);
        top--;
    }
}

// Display
void display() {
    if (top == -1)
        printf("Stack is empty\n");
    else {
        printf("Stack elements:\n");
        for (int i = top; i >= 0; i--)
            printf("%d\n", stack[i]);
    }
}

// Main
int main() {
    push(10);
    push(20);
    push(30);
    display();
    pop();
    display();
    return 0;
}

```

OUTPUT-



```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p3a.c -o p3a
PS D:\DS PROGRAMS\LABPROGRAMS> ./p3a
Pushed 10
Pushed 20
Pushed 30
Pushed 40
Popped 40
Stack elements:
30
20
10
PS D:\DS PROGRAMS\LABPROGRAMS> 

```

LAB PROGRAM 3B

```

#include <stdio.h>
#include <stdlib.h>

```

```

// Define node structure

```

```

struct Node {
    int data;
    struct Node* next;
};
struct Node* top = NULL; // Top of stack
// Push operation
void push(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));

    if (newNode == NULL) {
        printf("Stack Overflow (Memory not available)\n");
        return;
    }

    newNode->data = value;
    newNode->next = top;
    top = newNode;

    printf("%d pushed to stack\n", value);
}
// Pop operation
void pop() {
    if (top == NULL) {
        printf("Stack Underflow (Empty stack)\n");
        return;
    }
    struct Node* temp = top;
    printf("%d popped from stack\n", top->data);

    top = top->next;
    free(temp);
}

// Peek operation
void peek() {
    if (top == NULL) {
        printf("Stack is empty\n");
    } else {
        printf("Top element is: %d\n", top->data);
    }
}

// Display stack
void display() {
    if (top == NULL) {
        printf("Stack is empty\n");
    }
}

```

```

        return;
    }

    struct Node* temp = top;
    printf("Stack elements: ");

    while (temp != NULL) {
        printf("%d -> ", temp->data);
        temp = temp->next;
    }
    printf("NULL\n");
}

// Main function
int main() {
    int choice, value;

    while (1) {
        printf("\n--- Stack Menu ---\n");
        printf("1. Push\n2. Pop\n3. Peek\n4. Display\n5. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter value: ");
                scanf("%d", &value);
                push(value);
                break;

            case 2:
                pop();
                break;

            case 3:
                peek();
                break;

            case 4:
                display();
                break;

            case 5:
                exit(0);

            default:

```

```

        printf("Invalid choice\n");
    }
}

return 0;
}

```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc 3b.c -o 3b
PS D:\DS PROGRAMS\LABPROGRAMS> ./3b

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 1
Enter value to push: 80
80 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 1
Enter value to push: 20
20 pushed to stack

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 4
Stack elements: 20 -> 80 -> NULL

--- Stack Menu ---
1. Push
2. Pop
3. Peek
4. Display (Traverse)
5. Exit
Enter your choice: 5
Exiting...
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 4

#Converting an infix expression to postfix using a stack.

```

#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <ctype.h>

#define MAX 100

char stack[MAX];
int top = -1;

void push(char c) {
    if (top >= MAX - 1) {
        printf("Stack Overflow\n");
        return;
    }
    stack[++top] = c;
}

```



```

char pop() {
    if (top < 0) {
        return '\0';
    }
    return stack[top--];
}

char peek() {
    if (top < 0) {
        return '\0';
    }
    return stack[top];
}

int precedence(char op) {
    switch (op) {
        case '+':
        case '-':
            return 1;
        case '*':
        case '/':
            return 2;
        case '^':
            return 3;
        default:
            return 0;
    }
}

int isOperator(char c) {
    return (c == '+' || c == '-' || c == '*' || c == '/' || c == '^');
}

void infixToPostfix(char *infix, char *postfix) {
    int i = 0, j = 0;
    char c;
    while ((c = infix[i++]) != '\0') {
        // If operand, add to output
        if (isalnum(c)) {
            postfix[j++] = c;
        }
        // If '(', push to stack
        else if (c == '(') {
            push(c);
        }
        // If ')', pop until '(' is found
        else if (c == ')') {

```

```

        while (top >= 0 && peek() != '(') {
            postfix[j++] = pop();
        }
        pop(); // Remove '(' from stack
    }
    // If operator
    else if (isOperator(c)) {
        // Handle right-associativity for '^'
        if (c == '^') {
            while (top >= 0 && precedence(peek()) > precedence(c)) {
                postfix[j++] = pop();
            }
        } else {
            while (top >= 0 && precedence(peek()) >= precedence(c)) {
                postfix[j++] = pop();
            }
        }
        push(c);
    }
}
// Pop remaining operators
while (top >= 0) {
    postfix[j++] = pop();
}
postfix[j] = '\0';
}
int main() {
    char infix[MAX], postfix[MAX];
    printf("Enter an infix expression: ");
    scanf("%s", infix);
    infixToPostfix(infix, postfix);
    printf("Postfix expression: %s\n", postfix);
    return 0;
}

```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p4.c -o p4
PS D:\DS PROGRAMS\LABPROGRAMS> ./p4
Enter an infix expression: (A+B)*C
Postfix Expression: AB+C*
PS D:\DS PROGRAMS\LABPROGRAMS> 

```

LAB PROGRAM 5

Evaluating a postfix expression using a stack.

```
#include <stdio.h>
```

```
#include <ctype.h>
```

```
#define MAX 100
```

```
int stack[MAX];
```

```
int top = -1;
```

```
// Push operation
```

```
void push(int value)
```

```
{
```

```
    top++;
```

```
    stack[top] = value;
```

```
}
```

```
// Pop operation
```

```
int pop()
```

```
{
```

```
    return stack[top--];
```

```
}
```

```
int main()
```

```
{
```

```
    char postfix[MAX];
```

```
    int i, op1, op2, result;
```

```
    printf("Enter postfix expression: ");
```

```
    scanf("%s", postfix);
```

```
    for(i = 0; postfix[i] != '\0'; i++)
```

```
    {
```

```
        // If character is operand
```

```
        if(isdigit(postfix[i]))
```

```
        {
```

```
            push(postfix[i] - '0');
```

```
        }
```

```
        // If character is operator
```

```
        else
```

```
        {
```

```
            op2 = pop();
```

```
            op1 = pop();
```

```
            switch(postfix[i])
```

```

{
    case '+':
        result = op1 + op2;
        break;

    case '-':
        result = op1 - op2;
        break;

    case '*':
        result = op1 * op2;
        break;

    case '/':
        result = op1 / op2;
        break;

    case '%':
        result = op1 % op2;
        break;

    default:
        printf("Invalid operator");
        return 1;
}

push(result);
}
}

printf("Result = %d", pop());

return 0;
}

```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p5.c -o p5
PS D:\DS PROGRAMS\LABPROGRAMS> ./p5
Enter postfix expression: 23+
Result = 5
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 6A

#Queue operations using arrays

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int queue[MAX];
```

```
int front = -1, rear = -1;
```

```
// Insert element
```

```
void enqueue(int value)
```

```
{
```

```
    if(rear == MAX - 1)
```

```
    {
```

```
        printf("Queue Overflow\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        if(front == -1)
```

```
            front = 0;
```

```
        rear++;
```

```
        queue[rear] = value;
```

```
        printf("%d inserted\n", value);
```

```
    }
```

```
}
```

```
// Delete element
```

```
void dequeue()
```

```
{
```

```
    if(front == -1 || front > rear)
```

```
    {
```

```
        printf("Queue Underflow\n");
```

```
    }
```

```
    else
```

```
    {
```

```
        printf("%d deleted\n", queue[front]);
```

```
        front++;
```

```
    }
```

```
}
```

```
// Display queue
```

```
void display()
```

```
{
```

```

int i;

if(front == -1 || front > rear)
{
    printf("Queue is Empty\n");
}
else
{
    printf("Queue Elements:\n");

    for(i = front; i <= rear; i++)
    {
        printf("%d ", queue[i]);
    }

    printf("\n");
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    dequeue();

    display();

    return 0;
}

```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p6a.c -o p6a
PS D:\DS PROGRAMS\LABPROGRAMS> ./p6a
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 6B

#Queue operations using Linked List

```
#include <stdio.h>
#include <stdlib.h>

// Node structure
struct Node
{
    int data;
    struct Node *next;
};

struct Node *front = NULL;
struct Node *rear = NULL;

// Insert element
void enqueue(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->next = NULL;

    if(rear == NULL)
    {
        front = rear = newNode;
    }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }

    printf("%d inserted\n", value);
}

// Delete element
void dequeue()
{
    struct Node *temp;

    if(front == NULL)
```

```

    {
        printf("Queue Underflow\n");
    }
    else
    {
        temp = front;

        printf("%d deleted\n", front->data);

        front = front->next;

        if(front == NULL)
            rear = NULL;

        free(temp);
    }
}

// Display queue
void display()
{
    struct Node *temp;

    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        temp = front;

        printf("Queue Elements:\n");

        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }

        printf("\n");
    }
}

int main()
{
    enqueue(10);

```



```

    enqueue(20);
    enqueue(30);

    display();

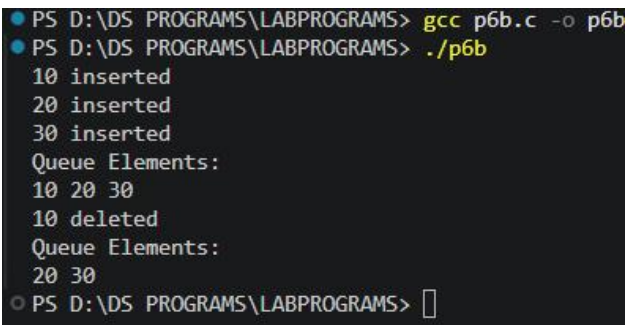
    dequeue();

    display();

    return 0;
}

```

OUTPUT-



```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p6b.c -o p6b
PS D:\DS PROGRAMS\LABPROGRAMS> ./p6b
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
PS D:\DS PROGRAMS\LABPROGRAMS> 

```

LAB PROGRAM 7

#Implementing a circular queue using arrays.

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
// Node structure
```

```
struct Node
```

```
{
```

```
int data;
```

```
struct Node *next;
```

```
};
```

```
struct Node *front = NULL;
```

```
struct Node *rear = NULL;
```

```
// Insert element
```

```
void enqueue(int value)
```

```
{
```

```
struct Node *newNode;  
newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
newNode->data = value;
```

```
newNode->next = NULL;
```

```
if(rear == NULL)
```

```
{
```

```
front = rear = newNode;
```

```
}
```

```
else
```

```
{
```

```
rear->next = newNode;
```

```
rear = newNode;
```

```
}
```

```
printf("%d inserted\n", value);
```

```
}
```

```
// Delete element
```

```
void dequeue()
```

```
{
```

```
struct Node *temp;
```

```
if(front == NULL)
```

```
{
```

```
printf("Queue Underflow\n");
```

```
}
```

```
else
```

```
{
```

```
temp = front;
```

```
printf("%d deleted\n", front->data);
```

```
front = front->next;
```

```
if(front == NULL)
```

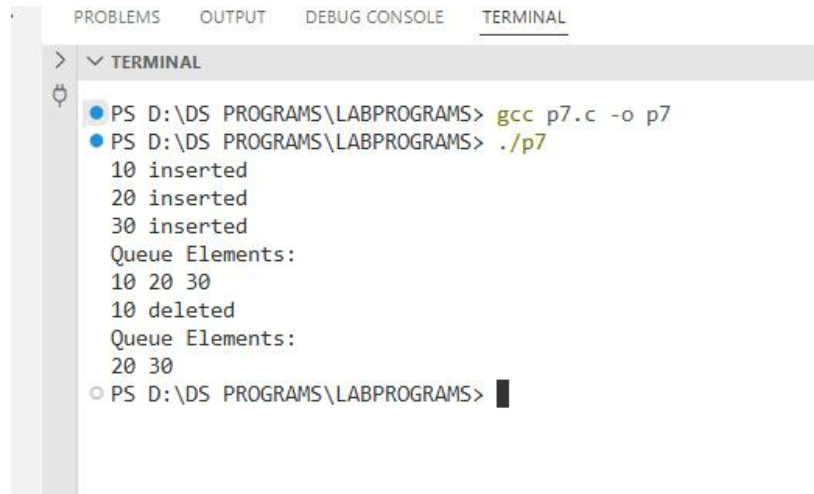
```
rear = NULL;
free(temp);
}
}
// Display queue
void display()
{
    struct Node *temp;
    if(front == NULL)
    {
        printf("Queue is Empty\n");
    }
    else
    {
        temp = front;
        printf("Queue Elements:\n");
        while(temp != NULL)
        {
            printf("%d ", temp->data);
            temp = temp->next;
        }
    }
    printf("\n");
}
int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);
```

```

display();
dequeue();
display();
return 0;
}

```

OUTPUT-



```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p7.c -o p7
PS D:\DS PROGRAMS\LABPROGRAMS> ./p7
10 inserted
20 inserted
30 inserted
Queue Elements:
10 20 30
10 deleted
Queue Elements:
20 30
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 8

#Implementing a priority queue using arrays or linked lists.

```
#include <stdio.h>
```

```
#define MAX 5
```

```
int pq[MAX];
```

```
int size = 0;
```

```
// Insert element in sorted order
```

```
void enqueue(int value)
```

```
{
```

```
int i;
```

```
if(size == MAX)
```

```
{
```

```
printf("Priority Queue Overflow\n");
```

```
return;
```

```

}
// Insert based on priority (smaller number = higher priority)
for(i = size - 1; i >= 0 && pq[i] > value; i--)
{
    pq[i + 1] = pq[i];
}
pq[i + 1] = value;
size++;
printf("%d inserted\n", value);
}
// Delete highest priority element
void dequeue()
{
    int i;
    if(size == 0)
    {
        printf("Priority Queue Underflow\n");
        return;
    }
    printf("%d deleted\n", pq[0]);
    for(i = 0; i < size - 1; i++)
    {
        pq[i] = pq[i + 1];
    }
    size--;
}
// Display elements
void display()
{
    int i;

```

```
if(size == 0)
{
printf("Priority Queue is Empty\n");
}
else
{
printf("Priority Queue Elements:\n");
for(i = 0; i < size; i++)
{
printf("%d ", pq[i]);
}
}
printf("\n");
}
int main()
{
enqueue(30);
enqueue(10);
enqueue(50);
enqueue(20);
display();
dequeue();
display();
return 0;
}
```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p8.c -o p8
PS D:\DS PROGRAMS\LABPROGRAMS> ./p8
30 inserted
10 inserted
50 inserted
20 inserted
Priority Queue Elements:
10 20 30 50
10 deleted
Priority Queue Elements:
20 30 50
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 9

#Implementing singly linked list operations: insertion, deletion, and search.

```

#include <stdio.h>
#include <stdlib.h>

```

```

struct Node {
    int data;
    struct Node* next;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

```

```

void insertAtBeginning(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    newNode->next = *head;
    *head = newNode;
}

```

```

void insertAtEnd(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }
}

```

```

    struct Node* temp = *head;
    while (temp->next != NULL) {
        temp = temp->next;
    }
    temp->next = newNode;
}

void insertAtPosition(struct Node** head, int data, int position) {
    struct Node* newNode = createNode(data);
    if (position == 0) {
        newNode->next = *head;
        *head = newNode;
        return;
    }
    struct Node* temp = *head;
    for (int i = 0; temp != NULL && i < position - 1; i++) {
        temp = temp->next;
    }
    if (temp == NULL) {
        printf("Position out of range\n");
        return;
    }
    newNode->next = temp->next;
    temp->next = newNode;
}

void deleteFromBeginning(struct Node** head) {
    if (*head == NULL) return;
    struct Node* temp = *head;
    *head = (*head)->next;
    free(temp);
}

void deleteFromEnd(struct Node** head) {
    if (*head == NULL) return;
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    struct Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    free(temp->next);
    temp->next = NULL;
}

```



```
}
```

```
void deleteFromPosition(struct Node** head, int position) {  
    if (*head == NULL) return;  
    struct Node* temp = *head;  
    if (position == 0) {  
        *head = temp->next;  
        free(temp);  
        return;  
    }  
    for (int i = 0; temp != NULL && i < position - 1; i++) {  
        temp = temp->next;  
    }  
    if (temp == NULL || temp->next == NULL) {  
        printf("Position out of range\n");  
        return;  
    }  
    struct Node* next = temp->next->next;  
    free(temp->next);  
    temp->next = next;  
}
```

```
void traverse(struct Node* head) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        printf("%d -> ", temp->data);  
        temp = temp->next;  
    }  
    printf("NULL\n");  
}
```

```
int search(struct Node* head, int target) {  
    struct Node* temp = head;  
    while (temp != NULL) {  
        if (temp->data == target) return 1;  
        temp = temp->next;  
    }  
    return 0;  
}
```

```
void reverse(struct Node** head) {  
    struct Node* prev = NULL;  
    struct Node* current = *head;  
    struct Node* next = NULL;  
    while (current != NULL) {  
        next = current->next;
```

```

        current->next = prev;
        prev = current;
        current = next;
    }
    *head = prev;
}

void sort(struct Node** head) {
    if (*head == NULL || (*head)->next == NULL) return;
    struct Node* temp = *head;
    while (temp != NULL) {
        struct Node* next = temp->next;
        while (next != NULL) {
            if (temp->data > next->data) {
                int data = temp->data;
                temp->data = next->data;
                next->data = data;
            }
            next = next->next;
        }
        temp = temp->next;
    }
}

int length(struct Node* head) {
    int count = 0;
    struct Node* temp = head;
    while (temp != NULL) {
        count++;
        temp = temp->next;
    }
    return count;
}

int main() {
    struct Node* head = NULL;

    insertAtEnd(&head, 10);
    insertAtEnd(&head, 20);
    insertAtEnd(&head, 30);
    traverse(head);

    insertAtBeginning(&head, 5);
    traverse(head);

    insertAtPosition(&head, 25, 2);

```

```

    traverse(head);

    deleteFromBeginning(&head);
    traverse(head);

    deleteFromEnd(&head);
    traverse(head);

    deleteFromPosition(&head, 1);
    traverse(head);

    printf("List contains 20: %s\n", search(head, 20) ? "Yes" : "No");
    printf("List contains 40: %s\n", search(head, 40) ? "Yes" : "No");

    reverse(&head);
    traverse(head);

    insertAtEnd(&head, 15);
    insertAtEnd(&head, 5);
    traverse(head);

    sort(&head);
    traverse(head);

    printf("Length of the list: %d\n", length(head));

    return 0;
}

    preorder(root->right);
}

// Inorder Traversal
void inorder(struct Node *root)
{
    if(root != NULL)
    {
        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Postorder Traversal

```

```

void postorder(struct Node *root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main()
{
    // Creating Binary Tree
    struct Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);

    root->left->left = createNode(4);
    root->left->right = createNode(5);

    // Traversals
    printf("Preorder Traversal:\n");
    preorder(root);

    printf("\n\nInorder Traversal:\n");
    inorder(root);

    printf("\n\nPostorder Traversal:\n");
    postorder(root);

    return 0;
}

```

OUTPUT-

```

PS D:\DS PROGRAMS\LABPROGRAMS> gcc p9.c -o p9
PS D:\DS PROGRAMS\LABPROGRAMS> ./p9
10->20->30->NULL
5->10->20->30->NULL
5->10->25->20->30->NULL
5->10->25->25->20->30->NULL
10->25->25->20->30->NULL
10->25->25->20->NULL
10->25->20->NULL
List contains 20:Yes
List contains 40:No
PS D:\DS PROGRAMS\LABPROGRAMS>

```

LAB PROGRAM 10

Implementing binary tree creation and traversal (preorder, inorder, postorder).

```
#include <stdio.h>
#include <stdlib.h>
```

```
// Structure for tree node
struct Node
{
    int data;
    struct Node *left;
    struct Node *right;
};
```

```
// Create new node
struct Node* createNode(int value)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->data = value;
    newNode->left = NULL;
    newNode->right = NULL;

    return newNode;
}
```

```
// Preorder Traversal
void preorder(struct Node *root)
{
    if(root != NULL)
    {
        printf("%d ", root->data);
        preorder(root->left);
        preorder(root->right);
    }
}
```

```
// Inorder Traversal
void inorder(struct Node *root)
{
    if(root != NULL)
    {
```

```

        inorder(root->left);
        printf("%d ", root->data);
        inorder(root->right);
    }
}

// Postorder Traversal
void postorder(struct Node *root)
{
    if(root != NULL)
    {
        postorder(root->left);
        postorder(root->right);
        printf("%d ", root->data);
    }
}

int main()
{
    // Creating Binary Tree
    struct Node *root = createNode(1);

    root->left = createNode(2);
    root->right = createNode(3);

    root->left->left = createNode(4);
    root->left->right = createNode(5);

    // Traversals
    printf("Preorder Traversal:\n");
    preorder(root);

    printf("\n\nInorder Traversal:\n");
    inorder(root);

    printf("\n\nPostorder Traversal:\n");
    postorder(root);

    return 0;
}
OUTPUT-
```

```
.. PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
> TERMINAL
PS D:\DS PROGRAMS\LABPROGRAMS> gcc p10.c -o p10
PS D:\DS PROGRAMS\LABPROGRAMS> ./p10
Preorder Traversal:
1 2 4 5 3

Inorder Traversal:
4 2 5 1 3

Postorder Traversal:
4 5 2 3 1
PS D:\DS PROGRAMS\LABPROGRAMS> █
```

LAB PROGRAM 11A

Representing a graph using adjacency matrix

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int main()
{
    int graph[MAX][MAX];
    int vertices, edges;
    int i, j, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    // Initialize matrix with 0
    for(i = 0; i < vertices; i++)
    {
        for(j = 0; j < vertices; j++)
        {
            graph[i][j] = 0;
        }
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
```

```

scanf("%d %d", &src, &dest);

graph[src][dest] = 1;
graph[dest][src] = 1; // Undirected graph
}

// Display adjacency matrix
printf("\nAdjacency Matrix:\n");

for(i = 0; i < vertices; i++)
{
    for(j = 0; j < vertices; j++)
    {
        printf("%d ", graph[i][j]);
    }

    printf("\n");
}

return 0;
}

```

OUTPUT-

```

> ▾ TERMINAL
❏
• PS D:\DS PROGRAMS\LABPROGRAMS> gcc p11a.c -o p11a
• PS D:\DS PROGRAMS\LABPROGRAMS> ./p11a
Enter number of vertices: 3
Enter number of edges: 2
Enter edge (source destination): 0 1
Enter edge (source destination): 1 2

Adjacency Matrix:
0 1 0
1 0 1
0 1 0
• PS D:\DS PROGRAMS\LABPROGRAMS> █

```

LAB PROGRAM 11b
#Using adjacency List

```

#include <stdio.h>
#include <stdlib.h>

```



```

#define MAX 10

// Node structure
struct Node
{
    int vertex;
    struct Node *next;
};

struct Node *adjList[MAX];

// Create new node
struct Node* createNode(int v)
{
    struct Node *newNode;

    newNode = (struct Node*)malloc(sizeof(struct Node));

    newNode->vertex = v;
    newNode->next = NULL;

    return newNode;
}

// Add edge
void addEdge(int src, int dest)
{
    struct Node *newNode;

    // Add edge src -> dest
    newNode = createNode(dest);
    newNode->next = adjList[src];
    adjList[src] = newNode;

    // Add edge dest -> src (Undirected graph)
    newNode = createNode(src);
    newNode->next = adjList[dest];
    adjList[dest] = newNode;
}

// Display adjacency list
void display(int vertices)
{
    int i;
    struct Node *temp;

```

```

printf("\nAdjacency List:\n");

for(i = 0; i < vertices; i++)
{
    printf("%d -> ", i);

    temp = adjList[i];

    while(temp != NULL)
    {
        printf("%d -> ", temp->vertex);
        temp = temp->next;
    }

    printf("NULL\n");
}

int main()
{
    int vertices, edges;
    int i, src, dest;

    printf("Enter number of vertices: ");
    scanf("%d", &vertices);

    // Initialize adjacency list
    for(i = 0; i < vertices; i++)
    {
        adjList[i] = NULL;
    }

    printf("Enter number of edges: ");
    scanf("%d", &edges);

    // Input edges
    for(i = 0; i < edges; i++)
    {
        printf("Enter edge (source destination): ");
        scanf("%d %d", &src, &dest);

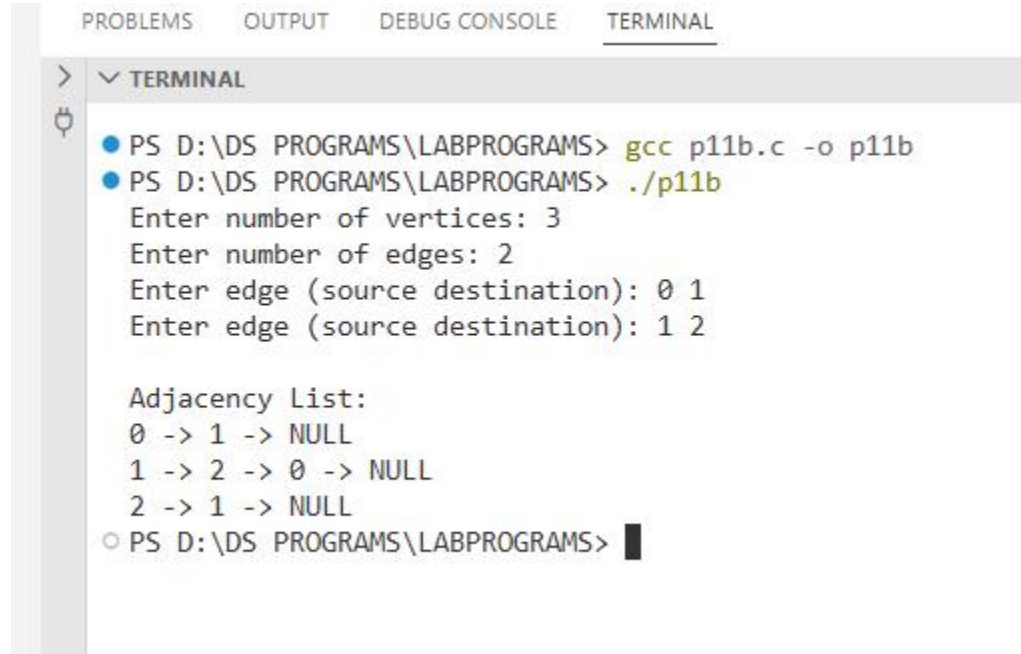
        addEdge(src, dest);
    }

    display(vertices);

```

```
    return 0;
}
```

OUTPUT-



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL
>  TERMINAL
❏
● PS D:\DS PROGRAMS\LABPROGRAMS> gcc p11b.c -o p11b
● PS D:\DS PROGRAMS\LABPROGRAMS> ./p11b
Enter number of vertices: 3
Enter number of edges: 2
Enter edge (source destination): 0 1
Enter edge (source destination): 1 2

Adjacency List:
0 -> 1 -> NULL
1 -> 2 -> 0 -> NULL
2 -> 1 -> NULL
○ PS D:\DS PROGRAMS\LABPROGRAMS> █
```

LAB PROGRAM 12

#Implementing graph traversal: BFS and DFS.

```
#include <stdio.h>
```

```
#define MAX 10
```

```
int graph[MAX][MAX];
int visited[MAX];
int queue[MAX];
int front = -1, rear = -1;
int vertices;
```

```
// Queue operations for BFS
```

```
void enqueue(int value)
{
    queue[++rear] = value;
}
```

```
int dequeue()
{
    return queue[++front];
}
```

```
// BFS Traversal
```

```

void BFS(int start)
{
    int i;

    printf("BFS Traversal: ");

    visited[start] = 1;
    enqueue(start);

    while(front != rear)
    {
        int node = dequeue();

        printf("%d ", node);

        for(i = 0; i < vertices; i++)
        {
            if(graph[node][i] == 1 && visited[i] == 0)
            {
                visited[i] = 1;
                enqueue(i);
            }
        }
    }
}

```

```

// DFS Traversal
void DFS(int start)
{
    int i;

    printf("%d ", start);

    visited[start] = 1;

    for(i = 0; i < vertices; i++)
    {
        if(graph[start][i] == 1 && visited[i] == 0)
        {
            DFS(i);
        }
    }
}

```

```

int main()
{

```

```

int edges;
int i, src, dest;
int start;

printf("Enter number of vertices: ");
scanf("%d", &vertices);

// Initialize graph
for(i = 0; i < vertices; i++)
{
    visited[i] = 0;

    for(int j = 0; j < vertices; j++)
    {
        graph[i][j] = 0;
    }
}

printf("Enter number of edges: ");
scanf("%d", &edges);

// Input edges
for(i = 0; i < edges; i++)
{
    printf("Enter edge (source destination): ");
    scanf("%d %d", &src, &dest);

    graph[src][dest] = 1;
    graph[dest][src] = 1; // Undirected graph
}

printf("Enter starting vertex: ");
scanf("%d", &start);

// BFS
BFS(start);

// Reset visited array for DFS
for(i = 0; i < vertices; i++)
{
    visited[i] = 0;
}

printf("\nDFS Traversal: ");

// DFS

```

```
DFS(start);  
  
return 0;  
}
```

OUTPUT-

```
DFS Traversal: 2 1 0  
● PS D:\DS PROGRAMS\LABPROGRAMS> gcc p12.c -o p12  
● PS D:\DS PROGRAMS\LABPROGRAMS> ./p12  
Enter number of vertices: 4  
Enter number of edges: 4  
Enter edge (source destination): 0 1  
Enter edge (source destination): 0 2  
Enter edge (source destination): 1 3  
Enter edge (source destination): 2 3  
Enter starting vertex: 0  
BFS Traversal: 0 1 2 3  
DFS Traversal: 0 1 3 2  
○ PS D:\DS PROGRAMS\LABPROGRAMS> █
```